

AI-Based Smart Object Tampering and Theft Detection System using Computer Vision and Arduino

Project Report

Abstract

This project presents an AI-based smart object tampering and theft detection system using Computer Vision and Arduino integration. The system continuously monitors an object using a webcam and compares the live camera frame with the initial reference frame. If a major scene change or object removal is detected, the system activates an alarm mechanism consisting of LEDs and a buzzer through Arduino communication.

The project combines Python, OpenCV, image processing, serial communication, and embedded systems to create a real-time anti-theft monitoring solution. The system is capable of detecting object tampering without requiring expensive industrial sensors.

Introduction

Security and object protection systems are becoming increasingly important in homes, jewelry stores, museums, offices, and industries. Traditional alarm systems usually depend on motion sensors, which often generate false alarms. This project introduces a smarter monitoring system using Computer Vision.

The system captures a reference frame containing the protected object. Whenever the object is removed or the monitored scene changes significantly, the system detects the tampering event and activates an alarm using Arduino.

This project demonstrates the integration of Artificial Intelligence concepts, image processing, embedded systems, and IoT fundamentals.

Objectives

- To develop a smart anti-theft monitoring system.
- To detect object tampering using Computer Vision.
- To integrate Python with Arduino using serial communication.
- To activate LED and buzzer alarms automatically.
- To reduce false alarms using contour area filtering.
- To create a low-cost real-time security system.

Components Required

Component	Quantity
Arduino UNO	1
Webcam / Laptop Camera	1
Red LED	1
Green LED	1
220Ω Resistor	2
Active Buzzer	1
Breadboard	1
Jumper Wires	Few
USB Cable	1

Software Requirements

Software	Purpose
Python	Main programming language
OpenCV	Computer Vision processing
Imutils	Image resizing utility
PySerial	Python-Arduino communication
Arduino IDE	Upload Arduino program
Python IDLE	Python program execution

Working Principle

1. The webcam captures live video frames.
2. The first frame is stored as the reference background.
3. Current frames are converted into grayscale and blurred.
4. The system compares the current frame with the reference frame.
5. If a significant difference is detected, contours are generated.
6. Small contour areas are ignored to reduce false alarms.
7. If the protected object is removed or the scene changes, the system sends a signal from Python to Arduino.
8. Arduino activates the buzzer and red LED.
9. During normal condition, the green LED remains ON.

Circuit Connections

Green LED

- Arduino Pin 12 → Green LED (+)
- Green LED (-) → 220Ω resistor → GND

Red LED

- Arduino Pin 13 → Red LED (+)
- Red LED (-) → 220Ω resistor → GND

Active Buzzer

- Arduino Pin 8 → Buzzer (+)
 - Buzzer (-) → GND
-

Arduino Program

```
char data;

void setup()
{
    pinMode(12, OUTPUT);

    pinMode(13, OUTPUT);

    pinMode(8, OUTPUT);

    Serial.begin(9600);
}

void loop()
{
    if(Serial.available())
    {
        data = Serial.read();

        if(data == '1')
        {
            digitalWrite(12, LOW);

            digitalWrite(13, HIGH);

            digitalWrite(8, HIGH);
        }
    }
}
```

```
        else if(data == '0')
        {
            digitalWrite(12, HIGH);

            digitalWrite(13, LOW);

            digitalWrite(8, LOW);
        }
    }
}
```

Python Program

```
import cv2
import time
import imutils
import serial

#arduino connection
arduino = serial.Serial('COM5', 9600)

time.sleep(5)

#camera start
cam = cv2.VideoCapture(0)

time.sleep(2)

firstFrame = None

#large object detection area
area = 15000

while True:

    _, img = cam.read()

    text = "Normal"

    #resize image
    img = imutils.resize(img, width=1000)

    #convert to gray
    grayImg = cv2.cvtColor(
        img,
        cv2.COLOR_BGR2GRAY
    )

    #smooth image
```

```

gaussianImg = cv2.GaussianBlur(
    grayImg,
    (31, 31),
    0
)

#capture first frame
if firstFrame is None:

    firstFrame = gaussianImg

    continue

#difference between first frame and current frame
imgDiff = cv2.absdiff(
    firstFrame,
    gaussianImg
)

#threshold
threshImg = cv2.threshold(
    imgDiff,
    50,
    255,
    cv2.THRESH_BINARY
)[1]

#dilation
threshImg = cv2.dilate(
    threshImg,
    None,
    iterations=4
)

#find contours
cnts = cv2.findContours(
    threshImg.copy(),
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

cnts = imutils.grab_contours(cnts)

tamperDetected = False

for c in cnts:

    #ignore small changes
    if cv2.contourArea(c) < area:

        continue

    tamperDetected = True

```

```

(x, y, w, h) = cv2.boundingRect(c)

#draw rectangle
cv2.rectangle(
    img,
    (x, y),
    (x + w, y + h),
    (0, 255, 0),
    2
)

#if object removed / scene changed
if tamperDetected == True:

    text = "OBJECT TAMPER DETECTED"

    arduino.write(b'1')

    print("ALARM")

else:

    text = "Normal"

    arduino.write(b'0')

#display text
cv2.putText(
    img,
    text,
    (10, 20),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (0, 0, 255),
    2
)

#show camera
cv2.imshow(
    "cameraFeed",
    img
)

#press q to quit
key = cv2.waitKey(1) & 0xFF

if key == ord("q"):

    break

#release camera
cam.release()

```

```
#close windows  
cv2.destroyAllWindows()
```

Advantages

- Low-cost implementation.
 - Real-time monitoring.
 - Smart anti-theft detection.
 - Reduced false alarms.
 - Easy hardware integration.
 - Expandable to IoT applications.
-

Applications

- Jewelry shop protection.
 - Museum artifact monitoring.
 - Bank locker monitoring.
 - Smart home security.
 - Office equipment protection.
 - Industrial object monitoring.
-

Future Enhancements

- Telegram or mobile alerts.
 - ESP32-CAM integration.
 - Cloud monitoring dashboard.
 - Face recognition.
 - Human detection using YOLO.
 - Battery-powered portable system.
-

Conclusion

The AI-Based Smart Object Tampering and Theft Detection System successfully detects scene changes and object removal using Computer Vision techniques. The integration of Python and Arduino enables real-time alarm activation using LEDs and buzzer alerts.

The project demonstrates practical implementation of Artificial Intelligence, Embedded Systems, IoT concepts, and Automation technologies in a real-world security application.

- Green LED ON during normal condition.
 - Red LED + buzzer ON during object tampering.
 - Bounding box displayed around detected changes.
-

Author

MANIKANDAN K

EEE Student | AI + Embedded Systems Enthusiast

kottaimanibolt@gmail. com

[LinkedIn](#) | [Portfolio](#) | [Github](#)